Linux网络--网络层MTU与IP分片

- Linux网络--网络层MTU与IP分片
- 1. MTU 是什么
  - 1.1 MTU 与完整帧长
  - 1.2 路径 MTU
- 2. 分片的基本规则
  - 2.1 谁负责
    - 分片可能发生在哪里
  - 2.2 分的是有效载荷
  - 2.3 三个关键字段
- 3. 判断是否分片
  - 3.1 字段之间的关系
- 4. 3000 字节数据报实例
  - 4.1 原始报文
  - 4.2 拆成三片
  - 4.3 片偏移为什么除以 8
  - 4.4 如果 MTU 不是 1500
  - 4.5 IP 首部在每个片段中重复
- 5. 接收方如何重组
  - 重组状态需要资源
  - 5.1 缺片会怎样
  - 5.2 重叠片段与异常输入
  - 5.3 接收方什么时候放弃
- 6. MSS、MTU 与 TCP
  - 6.1 MSS 计算
    - MSS 不等于 MTU
  - 6.2 MSS 协商
  - 6.3 路径 MTU 发现
  - 6.4 PMTU 黑洞
- 7. 为什么仍然要理解分片
  - 7.1 抓包时怎样观察分片
  - 7.2 为什么 UDP 更容易遇到分片
  - 7.3 查看接口 MTU
  - 7.4 路径 MTU 与隧道开销
  - 7.5 小包正常、大包失败
  - 7.6 UDP 报文长度估算

- 7.7 TCP 发送大小与 MSS
- 7.8 分片与滑动窗口的关系
- 7.9 IP 选项会进一步压缩有效载荷
- 7.10 IPv6 的差异

- 8. 本篇小结

Linux网络--网络层MTU与IP分片

系列导航

上一篇：Linux网络--数据链路层ARP地址解析与缓存
本篇：MTU 与 IP 分片
下一篇：Linux网络--应用层DNS、代理与内网穿透

以太网帧的数据区通常最多承载 1500 字节，而 IPv4 数据报最大可以接近 65535 字节。当网络层交付给链路层的数据超过当前链路 MTU 时，IPv4 可能需要在网络层分片，接收方再重组。

分片不是正常网络通信的首选路径。它会增加处理成本，并放大丢包风险，所以 TCP 会在连接建立和数据发送时尽量通过 MSS 与路径 MTU 发现规避分片。

1. MTU 是什么

MTU（Maximum Transmission Unit）表示某种链路一次能够承载的最大 IP 数据报长度。

以太网常见值：

```
MTU = 1500 字节
```

这里的 1500 指链路层有效载荷上限，也就是 IP 数据报最大长度，不包含以太网 14 字节帧头和 4 字节 FCS。

不同链路可能有不同 MTU。报文从 MTU 大的网络进入 MTU 小的网络时，如果 IP 数据报仍然过大，就需要由网络层处理。

1.1 MTU 与完整帧长

以太网标准 MTU 通常指 IP 数据报最大为 1500 字节。完整以太网帧还要加上：

```
14 字节帧头 + 1500 字节数据 + 4 字节 FCS = 1518 字节
```

如果启用 VLAN 标签，帧头还会更长。因此“MTU=1500”不等于“电线上只能有 1500 个字节”。

1.2 路径 MTU

从源到目的的路径可能由不同链路组成：

```
主机 MTU 1500
-> 以太网 MTU 1500
-> VPN 隧道 MTU 1400
-> 目标网络 MTU 1500
```

整条路径的最小值为 1400，称为路径 MTU（PMTU）。超过这个值的报文可能被丢弃、分片或触发 ICMP 反馈。

2. 分片的基本规则

2.1 谁负责

IPv4 网络层负责把过大的数据报拆成片段；重组通常由最终目的主机的 IP 层完成。

```
网络层分片
链路层只负责提出 MTU 上限
目的主机网络层重组
```

分片过程对 TCP 通常是透明的。TCP 不知道 IP 中间是否发生了分片，只知道自己的字节流和 ACK。

分片可能发生在哪里

IPv4 分片不一定只在源主机发生。如果源主机发出的大报文在后续链路超过较小 MTU，中间路由器也可能分片。

现代网络通常尽量做到：

- 源主机根据 MSS 和 PMTU 控制报文大小；
- 发送方设置 DF 探测路径；
- 尽量避免路由器在途中分片；
- 遇到 MTU 限制时由源端降低发送大小。

2.2 分的是有效载荷

一个 IP 数据报可以理解为：

```
IP 首部 + IP 有效载荷
```

分片主要拆有效载荷，但每个片段都要重新拥有完整的 IP 首部。

2.3 三个关键字段

字段	作用
16 位标识	同一原始数据报的分片共享同一个标识
3 位标志	包含保留位、DF、MF
13 位片偏移	该片段有效载荷在原始有效载荷中的位置，以 8 字节为单位

DF（Don't Fragment）为 1 时，路由器不能分片；如果报文超过 MTU，IPv4 路由器通常会丢弃并返回 ICMP“需要分片”信息。

MF（More Fragments）：

- `MF=1`：后面还有片段；
- `MF=0`：最后一个片段，或原始报文没有分片。

3. 判断是否分片

MF	片偏移	判断
0	0	原始数据报未分片

MF	片偏移	判断
1	0	第一片，后面还有
1	非 0	中间片
0	非 0	最后一片

只要 `MF=1` 或片偏移非 0，就可以确定报文属于分片集合。

3.1 字段之间的关系

一个片段是否完整需要同时满足：

- IP 首部可通过首部长度定位；
- `总长度 = 首部长度 + 本片有效载荷长度`；
- 片偏移乘以 8 得到原始有效载荷中的起点；
- `MF=1` 表示后面还有片段；
- `MF=0` 表示这是集合末尾；
- 所有片段共享同一个 16 位标识，并结合源地址、目的地址和协议号识别集合。

单看 16 位标识不够，因为不同发送方可能恰好生成相同 ID。

4. 3000 字节数据报实例

4.1 原始报文

```
IP 总长度：3000 字节
IP 首部：20 字节
原始有效载荷：2980 字节
链路 MTU：1500 字节
```

每片最多承载：

```
1500 − 20 = 1480 字节有效载荷
```

4.2 拆成三片

```
2980 = 1480 + 1480 + 20
```

分片	IP 总长度	有效载荷	MF	实际偏移	13 位字段值
第 1 片	1500	1480	1	0	0
第 2 片	1500	1480	1	1480	185
第 3 片	40	20	0	2960	370

每片都拥有独立的 IP 首部，且共享同一个 16 位标识，例如 `1234`。

4.3 片偏移为什么除以 8

13 位字段最大值为 `8191`。如果直接表示字节偏移，只能覆盖 8191 字节，无法表达最大 IPv4 数据报的偏移。

协议约定实际偏移必须按 8 字节对齐：

```
写入：实际偏移 / 8
读取：字段值 * 8
```

因此：

```
1480 / 8 = 185
2960 / 8 = 370
```

除最后一片外，各片有效载荷长度必须是 8 的整数倍，否则下一片的实际偏移无法精确存入 13 位字段。

4.4 如果 MTU 不是 1500

若 MTU 为 1400，每片最大有效载荷为：

```
1400 - 20 = 1380 字节
```

对于 2980 字节原始有效载荷：

```
1380 + 1380 + 220 = 2980
```

因此需要三片，最后一片只有 220 字节。片偏移仍必须按 8 字节单位对齐。

分片大小不能机械地固定为 1480，要由当前链路 MTU 和 IP 首部长度共同决定。

4.5 IP 首部在每个片段中重复

把 3000 字节原始报文分成三片后，总线上实际传输的 IP 字节数不是 3000：

```
1500 + 1500 + 40 = 3040 字节
```

额外 40 字节是后两个片段新增的 IP 首部。原始报文中只有一份首部，分片后每片都需要一份，因此分片会放大首部开销。

5. 接收方如何重组

接收方通常根据以下信息识别同一原始数据报：

```
源 IP + 目的 IP + 协议号 + 16 位标识
```

然后：

1. 收集属于同一标识的片段；
2. 按片偏移升序排列；
3. 检查片段数据区间是否连续；
4. 检查最后一片的 `MF=0`；
5. 重组完整 IP 有效载荷；
6. 去掉 IP 首部，交给上层协议。

重组状态需要资源

接收方必须为尚未收全的数据报保存：

- 已到达片段；
- 每个片段覆盖的区间；

- 已获得的总长度估计；
- 定时器；
- 内存缓冲区。

如果攻击者发送大量不完整片段，会占用重组队列。因此操作系统会限制重组时间和内存用量，并在超时后释放状态。

5.1 缺片会怎样

如果任意必需片段丢失：

- 目的 IP 层无法形成完整数据报；
- TCP 接收到的是数据缺口或始终收不到完整段；
- TCP 最终通过超时或重复 ACK 触发重传；
- 从传输层观察，表现为更大范围的数据需要重发。

另一方面，TCP 可能根本没有交付一个超过 MTU 的完整段。如果发送方已按 MSS 控制大小，IP 层通常不需要分片。因此看到 TCP 段大小不等于分片大小，必须同时看 IP 层的标识、MF 和偏移。

假设每片独立不丢概率为 99%，四片全部到达的概率为：

```
0.99^4 ≈ 0.9606
```

分片越多，整个原始数据报全部到达的概率越低。

5.2 重叠片段与异常输入

如果两个片段声称覆盖相同的原始区间，就可能出现重叠。合法发送方不会构造重叠片段，但恶意报文可能利用不同系统对重叠部分的处理差异。

现代系统通常采用保守策略：

- 丢弃异常重叠；
- 限制片段总数；
- 限制重组内存；
- 限制等待时间；

- 只接受合理大小范围内的片段。

分片处理属于协议栈安全边界，不只是简单地拼接缓冲区。

5.3 接收方什么时候放弃

重组不能无限等待。若某个片段长时间不到，接收方会：

1. 释放已缓存片段；
2. 放弃这个原始数据报；
3. 可能发送 ICMP Time Exceeded；
4. 让上层继续依靠自己的超时和重传处理。

因此分片丢失不一定马上产生应用错误，可能表现为一次较大的网络延迟后才重传。UDP 应用如果依赖单个完整数据报，则可能直接丢失整条业务消息。

6. MSS、MTU 与 TCP

6.1 MSS 计算

标准 IPv4 首部 20 字节，标准 TCP 首部 20 字节：

```
MSS = MTU - IP 首部 - TCP 首部
    = 1500 - 20 - 20
    = 1460 字节
```

MSS 表示 TCP 有效载荷最大长度，不包含 TCP 首部和 IP 首部。

MSS 不等于 MTU

例如：

```
MTU = 1500
IP 首部 = 20
TCP 首部 = 20
MSS = 1460
```

若 TCP 带有时间戳等选项，TCP 首部可能超过 20 字节；若 IPv4 存在选项，IP 首部也可能超过 20 字节。实际可用 MSS 会相应减小。

UDP 没有 MSS 这样的建连协商字段。应用发送过大的 UDP 数据报时，可能直接导致 IP 层分片，因此 UDP 应用应按路径 MTU 控制单个数据报大小。

6.2 MSS 协商

TCP 建连时通过选项交换各自可接收的 MSS。发送方不应发送超过对端接收 MSS 的 TCP 有效载荷，同时还要考虑路径 MTU。

MSS 选项中的数值反映本端愿意接收的 TCP 载荷上限。双方方向可以不同，不能简单理解成双方永远统一使用同一个值。

6.3 路径 MTU 发现

路径 MTU 是源到目的之间所有链路中最小的 MTU。现代 TCP 常使用：

- DF 标志禁止中间分片；
- 中间路由器在报文过大时返回 ICMP“需要分片”；
- 发送方降低报文大小并重新发送。

如果 ICMP 被错误过滤，路径 MTU 发现可能失败，表现为连接建立成功但大响应卡住。

6.4 PMTU 黑洞

PMTU 黑洞常表现为：

```
ping 小包正常
TCP 三次握手正常
访问小页面可能正常
下载大响应或上传文件时卡死
```

原因可能是中间设备需要 ICMP 反馈，但防火墙把 ICMP 全部过滤，源主机始终不知道应该降低报文大小。

排查时可以：

```
ping -M do -s 1472 example.com
ping -M do -s 1400 example.com
```

通过比较不同大小的 DF 探测，逐步推断路径 MTU。对 VPN、PPPoE 和隧道环境，尤其需要关注额外封装开销。

7. 为什么仍然要理解分片

正常网页和文件传输通常通过 MSS 控制避免 IP 分片，但分片知识仍然重要：

- UDP 大报文可能触发分片；
- 某些 ICMP 报文需要特殊处理；
- 部分隧道、VPN 和中间设备会降低 MTU；
- 抓包时需要按标识和偏移识别分片；
- 网络排错必须区分“单帧丢失”和“整个数据报重组失败”。

7.1 抓包时怎样观察分片

Wireshark 中通常会看到：

- `Identification` 相同；
- `More fragments` 标志；
- `Fragment offset` 字段；
- 后续出现重组提示，或提示缺少片段。

如果是 TCP over IP，抓包工具可能在完整重组后才显示 TCP 段。若只抓到第一片，TCP 解析可能不完整。

7.2 为什么 UDP 更容易遇到分片

TCP 在连接阶段知道 MSS，并持续调整发送大小；UDP 应用直接交付数据报，应用需要自行控制：

- IP 首部长度；
- UDP 首部 8 字节；
- 路径 MTU；
- 是否需要应用层分块。

因此大 UDP 数据报更容易发生 IPv4 分片，或者在某条较小 MTU 链路上被丢弃。

7.3 查看接口 MTU

Linux：

```
ip link
ip link show eth0
```

输出中的 `mtu 1500` 表示该接口当前 MTU。修改需要管理员权限：

```
sudo ip link set dev eth0 mtu 1400
```

修改 MTU 会影响所有经过该接口的 IP 流量，不能只把它理解成针对某个 TCP 连接的参数。

7.4 路径 MTU 与隧道开销

VPN、GRE、IPIP、VXLAN 等隧道会增加额外首部：

```
原始 IP 报文
-> 再封装外层隧道首部
-> 物理链路
```

这样物理接口 MTU 为 1500 时，隧道内部可用 MTU 可能小于 1500。若仍按普通 MSS 发送，隧道封装后可能超过外层 MTU。

常见处理包括：

- 降低隧道接口 MTU；
- 调整 TCP MSS clamp；
- 允许必要的 ICMP 错误通过；
- 在应用层控制 UDP 数据报大小。

7.5 小包正常、大包失败

这是 MTU 问题最典型的表现。排查时不要只 ping 小包：

```
ping -c 2 -s 1000 example.com
ping -M do -c 2 -s 1472 example.com
ping -M do -c 2 -s 1400 example.com
```

逐步改变大小并观察何时失败，可以帮助定位路径 MTU。TCP 环境还可检查：

```
tcpdump -ni eth0 'icmp or tcp'
```

重点观察是否出现 ICMP Fragmentation Needed，以及 TCP 重传。

7.6 UDP 报文长度估算

UDP 报文结构为：

```
IPv4 首部 20 字节
UDP 首部 8 字节
UDP 数据 N 字节
```

在 MTU 1500、无 IP 选项时，一个不分片的 UDP 数据报最大数据长度为：

```
1500 - 20 - 8 = 1472 字节
```

因此：

```
UDP 数据 <= 1472：理论上可放进一个 1500 字节的 IPv4 报文
UDP 数据 > 1472：IPv4 可能需要分片
```

若路径 MTU 只有 1400，则安全上限要再降低：

```
1400 - 20 - 8 = 1372 字节
```

但该计算结果只适用于没有额外隧道和选项的场景。实际应用应留出安全余量。

7.7 TCP 发送大小与 MSS

如果两端 MSS 为 1460，TCP 每个数据段通常最多携带 1460 字节应用数据。到了 IP 层：

```
20 字节 IP 首部 + 20 字节 TCP 首部 + 1460 字节数据 = 1500 字节
```

恰好符合 MTU 1500。

如果路径中有 1400 MTU 的 VPN，而 TCP 仍发送 1460 字节有效载荷，IP 包就会变成 1500 字节，超过隧道内部承载能力。正确做法包括降低 MSS/MTU、在隧道边界进行 MSS clamp，或让 PMTU 发现完成调整。

7.8 分片与滑动窗口的关系

TCP 滑动窗口限制未确认数据量，MSS 限制单个 TCP 段大小，IP MTU 限制网络层数据报大小。三者回答的问题不同：

```
滑动窗口：还能发送多少未确认数据
MSS：一个 TCP 段一次最多带多少数据
MTU：一个链路层帧最多承载多大的 IP 报文
```

分片不是替代滑动窗口的机制，也不能修复窗口和拥塞控制问题。

7.9 IP 选项会进一步压缩有效载荷

标准首部为 20 字节，但 IPv4 允许选项把首部扩展到最多 60 字节。存在选项时，同样 MTU 下每片能够承载的数据会减少：

```
MTU 1500 − IP 首部 40 = 1460 字节
```

因此分析历史协议、诊断工具和特殊报文时，不能始终假设 IP 首部恰好 20 字节。

7.10 IPv6 的差异

IPv6 把分片相关字段放到扩展首部中，通常由源端分片，中间路由器不负责分片。IPv6 更依赖路径 MTU 发现和 ICMPv6 的 Packet Too Big。

IPv6 的最小链路 MTU 要求为 1280 字节。隧道和虚拟网络仍可能进一步降低可用值，因此 IPv6 应用同样不能假设端到端永远是 1500。

8. 本篇小结

```
MTU 限制一次链路能够承载的 IP 数据报大小
IPv4 网络层可以在必要时分片
目的主机按标识和偏移重组
片偏移以 8 字节为单位存储
分片增加开销和整体丢失概率
MSS 和路径 MTU 发现用于尽量避免分片
```

到这里，“IP 地址定位 → 路由找下一跳 → ARP 找 MAC → 帧受 MTU 限制”这条基础交付链已经完整。下一篇转到应用层的代表性问题：代理服务器为什么也能代表客户端或服务器转发请求。